嵌入式操作系统

Embedded Operating System

信息与软件工程学院

桑　楠

sn@uestc.edu.cn

2026年3月

第四部分

编写内核

——事务处理机制

核心内容

- 嵌入式内核的基本构成
- 任务管理
- 事件处理机制
- 内存管理机制
- 任务交互机制

事件处理机制

基本思想

- aCoral的任务执行次序与时间　由acoral_sched()决定
- 依据　调度策略
- 触发方式　中断、异常、时钟、… → 事件
- 事件处理机制　事件触发机制(Event-triggered)、时间触发(Time-triggered)机制

事件处理机制

中断

发展历程

- **最初**：被用来替换I/O操作的轮询处理方式，以提高I/O处理的效率。
- **包含**：中断包含了**自陷**（trap，也称为内部中断或是软件中断）的功能。
- **扩大**：中断的概念得到进一步扩大，被定义为导致程序正常执行流程发生改变的**事件**（不包括程序的分支情况）。可把概念被扩大的中断称为**广义中断**。

广义中断：中断、自陷、异常

- **中断**：由于CPU外部的原因而改变程序执行流程的过程。属于**异步事件**，又称为**硬件中断**。自陷和异常则为**同步事件**。
- **自陷**：通过处理器所拥有的软件指令、可预期地使处理器正在执行的程序流程发生变化，以执行特定的程序。自陷是显式的事件，需要无条件地执行；如Trap、SWI（**软中断**）、INT等指令。
- **异常**：为CPU自动产生的自陷，以处理异常事件。如被0除、执行非法指令和内存保护故障等。

分类

- **按可否屏蔽**：可屏蔽中断和不可屏蔽中断。
- **按中断源**：硬件中断和软件中断。
- **按中断信号的产生方式**：边缘触发中断和电平触发中断。
- **按中断服务程序的调用方式**：向量中断、直接中断和间接中断

中断处理

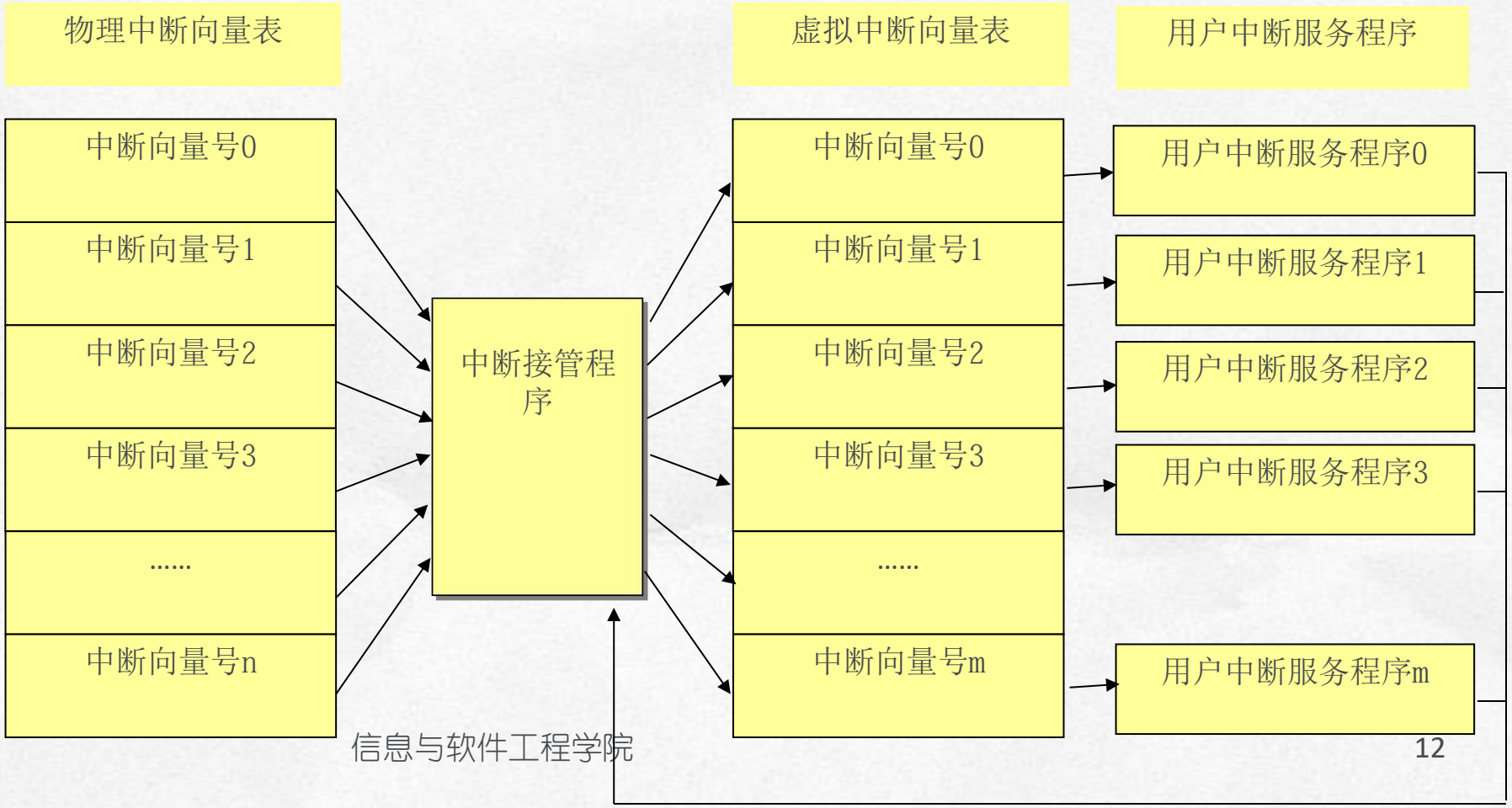
- **执行中断服务程序**　尽管导致自陷、异常和中断的事件不同，但大都具有相同的中断服务程序结构
- **中断服务程序的主要内容**
  - 保存上下文　保存中断服务程序**将要使用的**所有寄存器的内容，用于退出之前进行恢复现场
  - 多设备共享中断　为确定产生该中断信号的设备，需要**轮询**这些设备的中断状态寄存器
  - 具体的中断处理
  - 恢复保存的上下文
  - 执行中断返回指令，恢复被中断程序执行

中断处理 —— μCOS示例模板

```
YourISR:
    Save all CPU registers;                                    //保存环境
    Call OSIntEnter() or, increment OSIntNesting directly;     //是否嵌套
    if (OSIntNesting == 1) {
        OSTCBCur->OSTCBStkPtr = SP;
    }
    Clear interrupting device;
    Re-enable interrupts (optional)                            //允许高优先级中断
    Execute user code to service ISR;                          //中断程序执行
    Call OSIntExit();
    Restore all CPU registers;                                 //恢复环境
    Execute a return from interrupt instruction;
```

RTOS内核的中断管理

- **发生中断**：**中断接管程序**获得控制权，先进行处理；然后将控制权交给相应的用户中断服务程序
- **中断完成**：回到中断接管程序



实时系统的中断处理方式

- 三种方式
  - 中断作为前台任务　优先级高于所有软件，直接中断处理代码
  - 中断作为系统调用　优先级高于所有软件，可替换的中断服务程序（函数）
  - 中断作为任务切换　优先级与实时任务统一考虑，作为任务的中断服务程序

aCoral

中断处理

ARM的中断向量表

- ARM的IRQ —— 异常向量表中第七条

0x18	IRQ
0x14	(Reserved)
0x10	Data Abort
0x0C	Prefetch Abort
0x08	Software Interrupt
0x04	Undefined Instruction
0x00	Reset

(a) 异常向量表的内存地址

0x18	LDR pc, VECTOR_TABLE+18
0x14	(Reserved)
0x10	LDR pc, VECTOR_TABLE+10
0x0C	LDR pc, VECTOR_TABLE+C
0x08	LDR pc, VECTOR_TABLE+8
0x04	LDR pc, VECTOR_TABLE+4
0x00	LDR pc, VECTOR_TABLE+0

(b)异常向量表 的指令

中断向量表起始地址
VECTOR_TABLE

ARM的中断向量表（2）

- 执行过程 中断请求IRQ → 中断控制器 → 中断向量 → 中断向量表

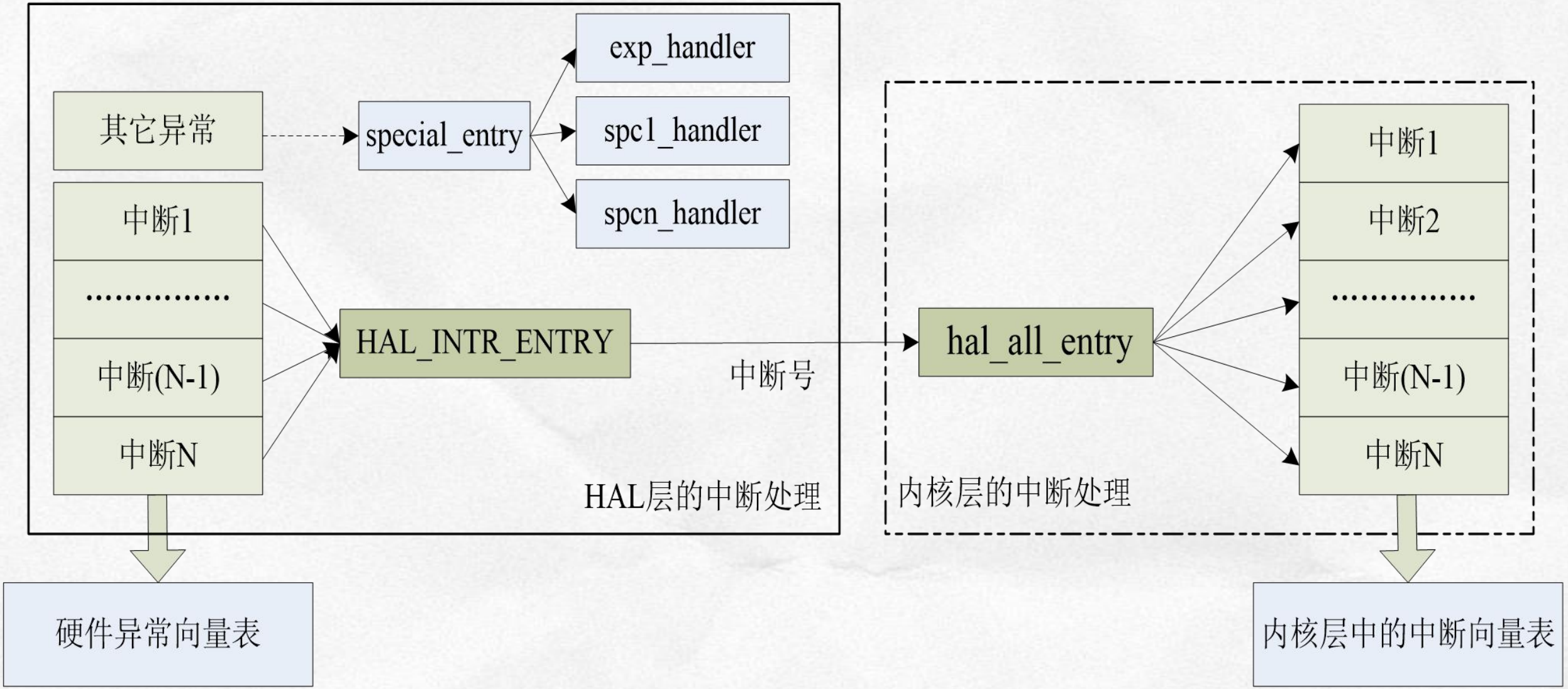
```
HAL_VECTR_START:
        LDR     pc, VECTOR_TABLE+0      @ Reset
        LDR     pc, VECTOR_TABLE+4      @ Undefined
        LDR     pc, VECTOR_TABLE+8      @ SWI
        LDR     pc, VECTOR_TABLE+0xc    @ Prefetch Abort
        LDR     pc, VECTOR_TABLE+0x10   @ Data Abort
        LDR     pc, VECTOR_TABLE+0x14   @ RESERVED
        LDR     pc, VECTOR_TABLE+0x18   @ IRQ
        LDR     pc, VECTOR_TABLE+0x1c   @ FIQ

VECTOR_TABLE:
            .long EXP_HANDLER
            .long EXP_HANDLER
            .long EXP_HANDLER
            .long EXP_HANDLER
            .long EXP_HANDLER
            .long EXP_HANDLER
            .long HAL_INTR_ENTRY
            .long EXP_HANDLER
HAL_VECTR_END:
```

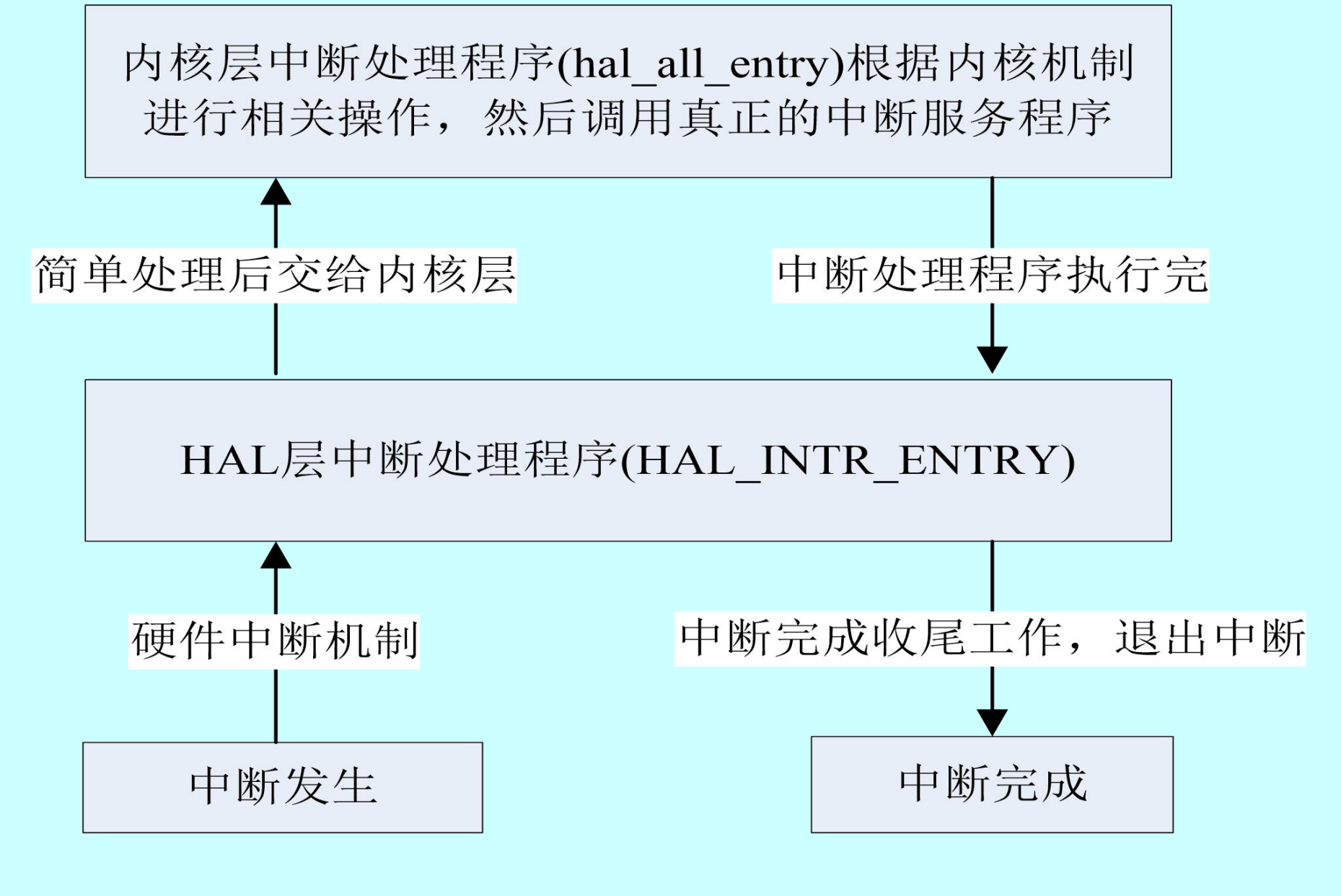
aCoral的中断处理模式

- 三种模式（类型）
  - 实时中断（通用） 直接调用中断处理程序，不经过：HAL→hal_all→intr→中断处理程序
  - 专家中断（专用，异常） 需要用户在中断处理程序中自己处理中断响应相关的操作，如清中断位等：特殊中断
  - 普通中断 不需要任何中断硬件相关的操作，只需编写中断要做什么

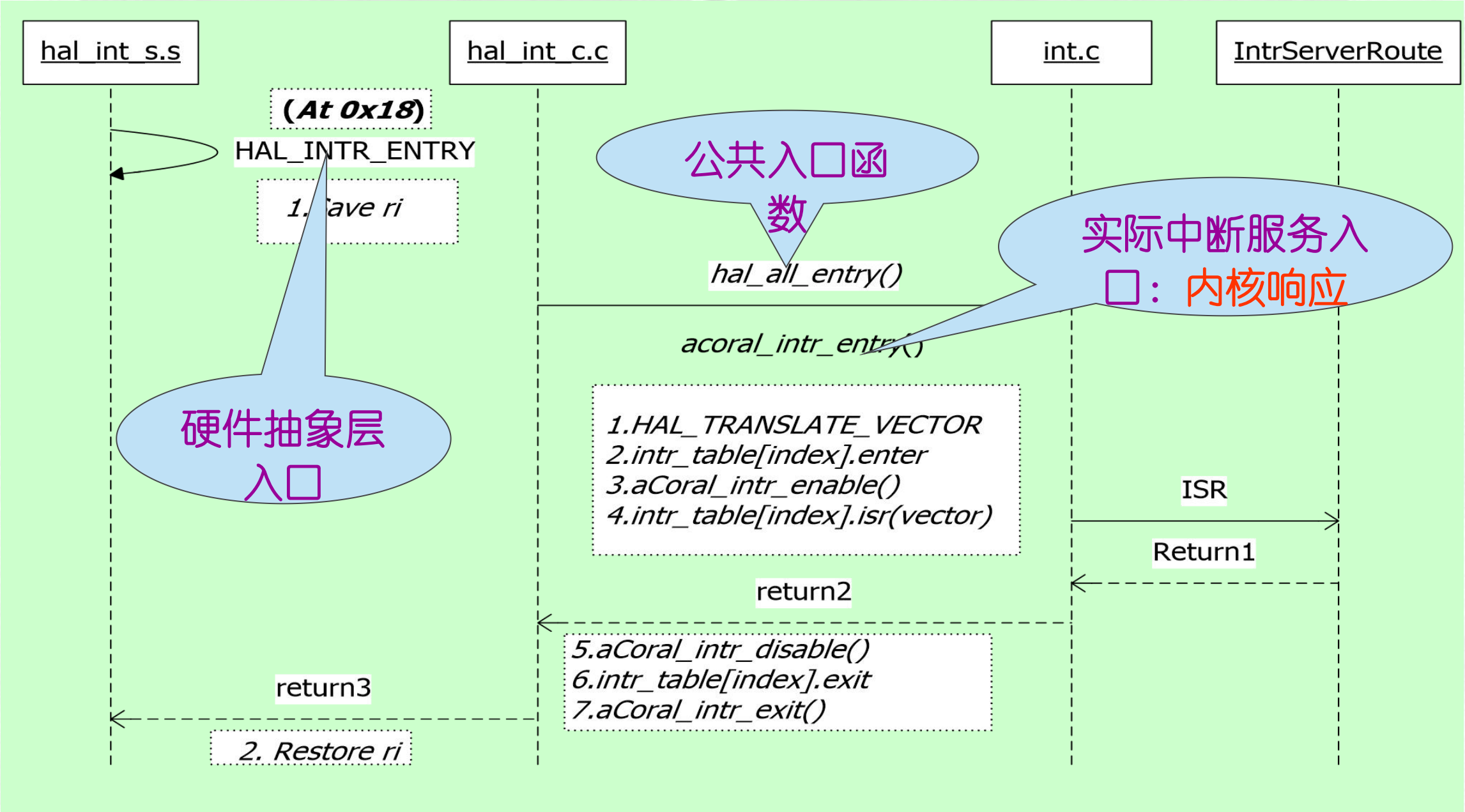
中断子程序逻辑结构


exp_handler
其它异常
special_entry
spc1_handler
spcn_handler
中断1
……
HAL_INTR_ENTRY
中断号
中断(N-1)
中断N
HAL层的中断处理
hal_all_entry
中断1
中断2
……
中断(N-1)
中断N
内核层的中断处理
硬件异常向量表
内核层中的中断向量表

中断处理流程


内核层中断处理程序(hal_all_entry)根据内核机制进行相关操作，然后调用真正的中断服务程序
简单处理后交给内核层
中断处理程序执行完
HAL层中断处理程序(HAL_INTR_ENTRY)
硬件中断机制
中断完成收尾工作，退出中断
中断发生
中断完成

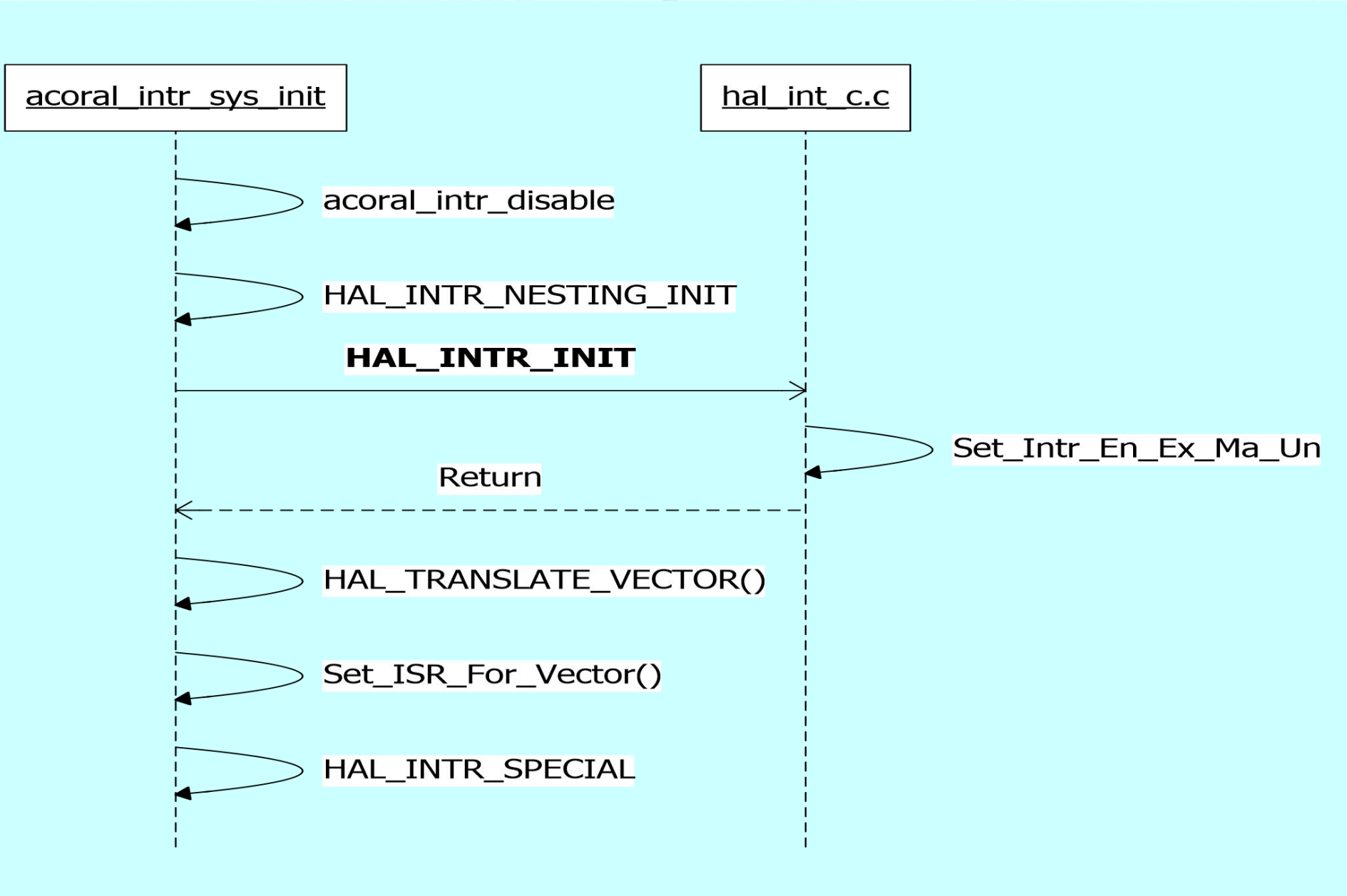
中断处理流程（2）


hal_int_s.s
hal_int_c.c
int.c
IntrServerRoute
(At 0x18)
HAL_INTR_ENTRY
1. Save ri
公共入口函数
hal_all_entry()
实际中断服务入口：内核响应
acoral_intr_entry()
硬件抽象层入口
1.HAL_TRANSLATE_VECTOR
2.intr_table[index].enter
3.aCoral_intr_enable()
4.intr_table[index].isr(vector)
ISR
Return1
return2
5.aCoral_intr_disable()
6.intr_table[index].exit
7.aCoral_intr_exit()
return3
2. Restore ri

aCoral的中断数据结构

```
typedef struct {
        acoral_u32 index;                    //内核层中断号(Index)
        acoral_u8 state;                     //中断状态
        acoral_u8 type;                      //中断类型
        void (*isr)(acoral_vector);          //ISR
        void (*enter)(acoral_vector);        //中断进入的处理：清中断位
        void (*exit)(acoral_vector);         //中断退出的处理
        void (*mask)(acoral_vector);         //中断屏蔽：不同CPU可不同
        void (*unmask)(acoral_vector);       //中断使能
}acoral_intr_ctr_t;
```

中断初始化：处理流程


acoral_intr_sys_init
hal_int_c.c
acoral_intr_disable
HAL_INTR_NESTING_INIT
HAL_INTR_INIT
Set_Intr_En_Ex_Ma_Un
Return
HAL_TRANSLATE_VECTOR()
Set_ISR_For_Vector()
HAL_INTR_SPECIAL

中断初始化

```
void acoral_intr_sys_init(){
    acoral_u32 i;
    acoral_vector index;
    acoral_intr_disable();                    //关中断
    HAL_INTR_NESTING_INIT();                  //中断嵌套标志初始化
    HAL_INTR_INIT();                          //中断底层硬件初始化
    for(i=HAL_INTR_MIN;i<=HAL_INTR_MAX;i++){   //每个中断的设置
        HAL_TRANSLATE_VECTOR(i,index);        // HAL层的中断号变成内核层中断号
        intr_table[index].isr=acoral_default_isr;     // ISR初始化
        intr_table[index].type=ACORAL_COMM_INTR;   // 中断类型初始化
        acoral_intr_mask(i);                  // 屏蔽该中断
    }
    HAL_INTR_SPECIAL();                       // 特殊中断初始化
}
```

中断嵌套初始化

```
void hal_intr_nesting_init_comm(){
        acoral_u32 i;
        for(i=0;i<HAL_MAX_CPU;i++)
                intr_nesting[i]=0;          // 对每个CPU，嵌套为0
}
```

HAL层初始化

```
void hal_intr_init(){
        acoral_u32 i;
        rPRIORITY = 0x00000000;          // 使用默认的固定优先级
        rINTMOD = 0x00000000;            // 所有中
        rEINTMSK = 0xffffffff;           //屏蔽所
        rINTMSK = 0xffffffff;            //屏蔽所有
        for (i=HAL_INTR_MIN;i<=HAL_INTR_MA  ,i++){
                acoral_set_intr_enter(i,hal_intr_ack);      //
                acoral_set_intr_exit(i,NULL);
                acoral_set_intr_mask(i,hal_intr_mask);      //
                acoral_set_intr_unmask(i,hal_intr_unmask); //打
        }
}
```

进入中断：依赖于不同的CPU

设置各中断的私有函数：具体代码属于内核层初始化

内核层初始化

- 对应HAL层的各函数初始化：硬件相

HAL中断号

```
// 中断进入初始化
void acoral_set_intr_enter(acoral_vector vector,void (*isr)(acoral_vector)){
        acoral_vector index;
        HAL_TRANSLATE_VECTOR(vector,index);
        intr_table[index].enter=isr;   // 设置中断进入函数hal_intr_ack
}
```

```
// 中断退出初始化
void acoral_set_intr_exit(acoral_vector vector,void (*isr)(acoral_vector)){
        acoral_vector index;
        HAL_TRANSLATE_VECTOR(vector,index);
        intr_table[index].exit=isr;      // 设置中断退出函数NULL
}
```

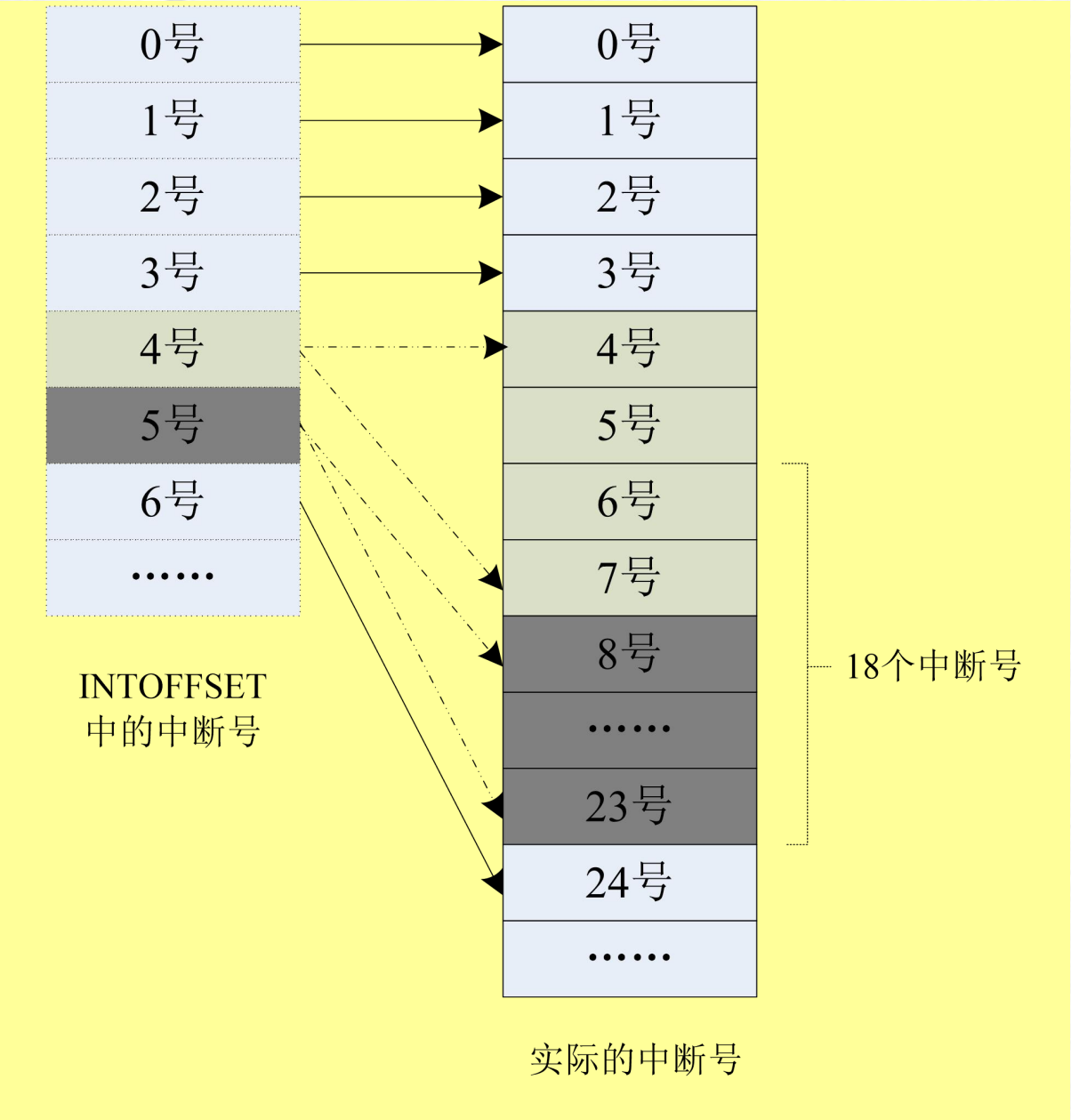
内核层初始化（2）

```
// 中断屏蔽初始化
void acoral_set_intr_mask(acoral_vector vector,void (*isr)(acoral_vector)){
        acoral_vector index;
        HAL_TRANSLATE_VECTOR(vector,index);
        intr_table[index].mask=isr;    // 设置中断屏蔽函数hal_intr_mask
}
```

```
// 中断使能初始化
void acoral_set_intr_unmask(acoral_vector vector,void (*isr)(acoral_vector)){
        acoral_vector index;
        HAL_TRANSLATE_VECTOR(vector,index);
        intr_table[index].unmask=isr;// 设置中断使能函数hal_intr_unmask
}
```

ARM2440的中断复用

- 内核层的中断号（实际）可以多于HAL的中断号（7个）；也可以少于HAL中断号——特殊使用占有
- 2440有24个GPIO输入触外部中断

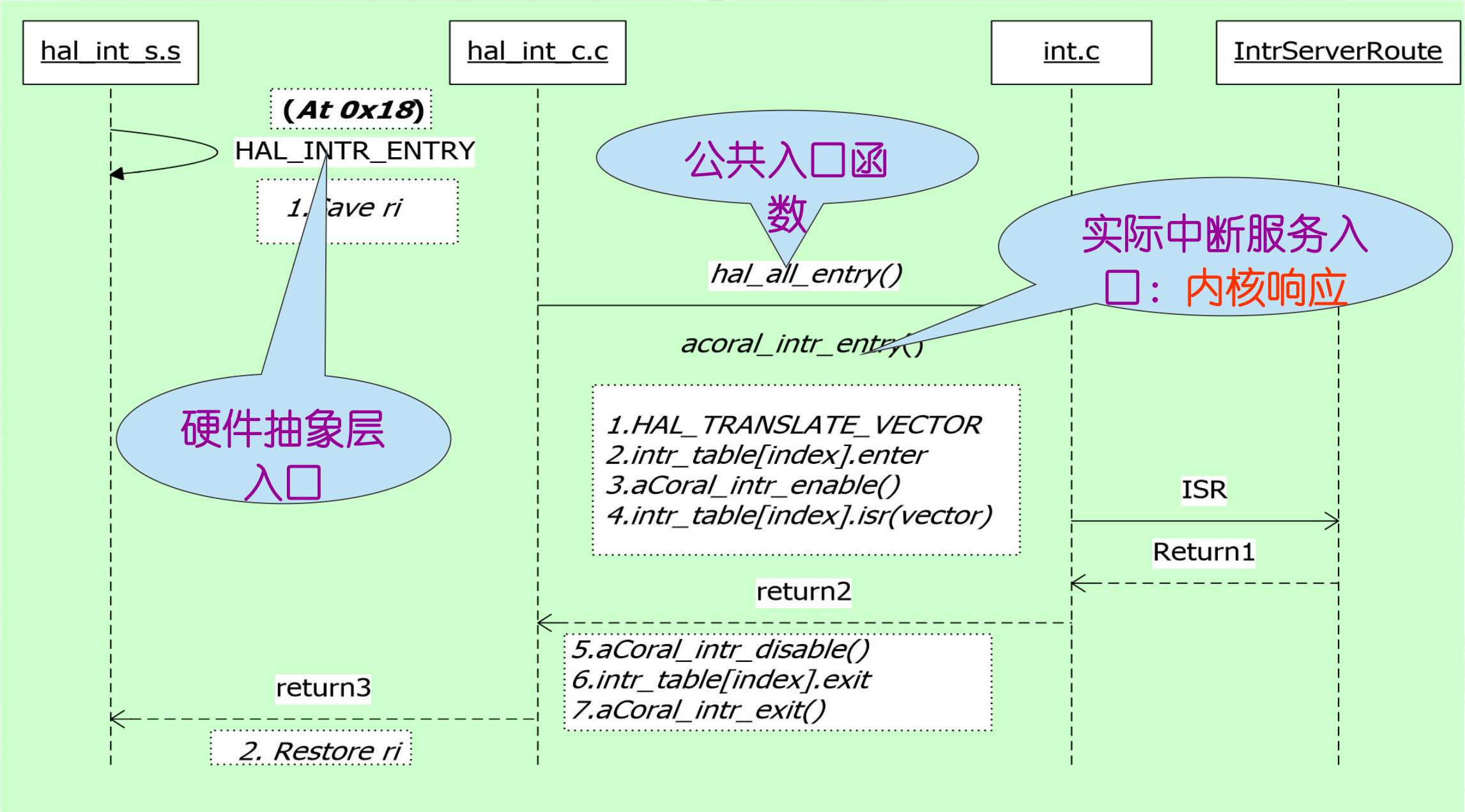


中断进入操作：按不复用设置

```
static void hal_intr_ack(acoral_u32 vector){ // ARM2440
    if((vector>3) && (vector<8)){            // 4~7号中断
        rEINTPND &= ~(1<<vector);            // 设置偏移的
        vector = 4;
    }
    else if((vector>7) && (vector<24)){      // 8~23号中断：中断号5
        rEINTPND &= ~(1<<vector);
        vector = 5;
    }
    else if(vector > 23)                     //24号以上中断：中断号6以上
        vector -= 18;
    rSRCPND = 1<<vector;          // 源未决寄存器设置：按中断号偏移
    rINTPND = 1<<vector;          // 中断未决寄存器设置：0~6
}
```

外部中断未决寄存器：用来区分共享中断4_7、8_23具体哪个触发

中断处理流程（2）


hal_int_s.s
hal_int_c.c
int.c
IntrServerRoute
(At 0x18)
HAL_INTR_ENTRY
1. Save ri
公共入口函数
hal_all_entry()
实际中断服务入口：内核响应
acoral_intr_entry()
硬件抽象层入口
1.HAL_TRANSLATE_VECTOR
2.intr_table[index].enter
3.aCoral_intr_enable()
4.intr_table[index].isr(vector)
ISR
Return1
return2
5.aCoral_intr_disable()
6.intr_table[index].exit
7.aCoral_intr_exit()
return3
2. Restore ri

硬件抽象层：入口

- HAL_INTR_ENTRY

```
HAL_INTR_ENTRY:
    stmfd   sp!,    {r0-r12,lr}          @保护通用寄存器及PC
    mrs     r1,     spsr
    stmfd   sp!,    {r1}                 @保护spsr，以支持中断嵌套
    msr     cpsr_c, #SVCMODE|NOIRQ       @进入SVCMODE,以便允许中断嵌套
    stmfd   sp!,    {lr}                 [illegible]用寄存器lr
    ldr     r0,     =INTOFFSET
    ldr     r0,     [r0]                 [illegible]断源（号）
    mov     lr,     pc                   @求得最新lr的值
    ldr     pc,     =hal_all_entry       @公共入口：r0代表传给C函数的第1参数
[illegible]
    ldmfd   sp!,    {lr}                 @恢复svc模式下的lr,
    msr     cpsr_c,#IRQMODE|NOINT        @更新cpsr,进入IRQ模式并禁止中断
    ldmfd   sp!,{r0}                     @spsr->r0
    msr     spsr_cxsf,r0                 @恢复spsr
    ldmfd   sp!,{r0-r12,lr}
    subs    pc,lr,#4                     @中断被重新打开
```

保护现场

偏移寄存器：中断源对应的唯一值

处理中断

恢复现场

硬件抽象层：中断开/关

```
HAL_INTR_ENABLE:
    mrs r0, cpsr
    bic r0, r0, #NOINT
    msr cpsr_cxsf, r0
    mov pc,lr
```

位清除命令：
按位逻辑与

```
HAL_INTR_DISABLE:
    mrs r0, cpsr
    mov r1, r0
    orr r1, r1, #NOINT
    msr cpsr_cxsf, r1
    mov pc , lr
```

中断公共入口函数

- hal_all_entry

  内核层响应

  需要判别实际

  的中断号

- rEINTPND

  外部中断未决寄

  存器　对应当

  前使用中断的

  位置一

```
void hal_all_entry(acoral_vector vector){
    unsigned long eint;
    unsigned long irq=4;
    if(vector==4||vector==5){
            eint=rEINTPND;              // EINTPND寄存器
            for(;irq<24;irq++){         // 移位操作：区分中断源
                    if(eint & (1<<irq)){
                    acoral_intr_entry(irq);
                    return;
            }
            }
    }
    if(vector>5)
            vector+=18;                         // 24号
    if(vector==4)
            acoral_prints("DErr\n");
    acoral_intr_entry(vector);                  // 实际中断服务入口
}
```

中断的内核层响应：实际入口

```
void acoral_intr_entry(acoral_vector vector){
    acoral_vector index;

    // HAL层中断号变为内核层中断表号
    HAL_TRANSLATE_VECTOR(vector,index);
    acoral_intr_nesting_inc();                    // 中断嵌套层+1
    if(intr_table[index].type==ACORAL_EXPERT_INTR){
        // 根据内核层中新中断号，找到对应的中断向量，处理
        intr_table[index].isr(vector);
        acoral_intr_disable();      }
    else{
```

中断的内核层响应：实际入口（2）

```
else{                    //之前hal_all中可能已关中断
    if(intr_table[index].enter!=NULL)      // 需要入口处理
        intr_table[index].enter(vector);
    acoral_intr_enable();                  // 开中断：增加调度点
    intr_table[index].isr(vector);         // 调用该中断的服务处理函数
    acoral_intr_disable();                 // 关中断
    if(intr_table[index].exit!=NULL)       // 需要出口处理
        intr_table[index].exit(vector);    // 此时中断表未空
}
acoral_intr_nesting_dec();                 // 本次终端待结束，嵌套减一
acoral_intr_exit();                        // 退出中断：调度
}
```

中断退出的处理

```
void acoral_intr_exit(){                    // 类似acoral_sched
        if(!acoral_need_sched)              // 调度标记为F
            return;
        if(acoral_intr_nesting)             // 属于嵌套中断
            return;
        if(acoral_sched_is_lock)            // 锁调度
            return;
        if (!acoral_start_sched)            // 开始调度标记未打开
            return;
        HAL_INTR_EXIT_BRIDGE();             // 退出时调度
}
```

事件处理机制

时钟

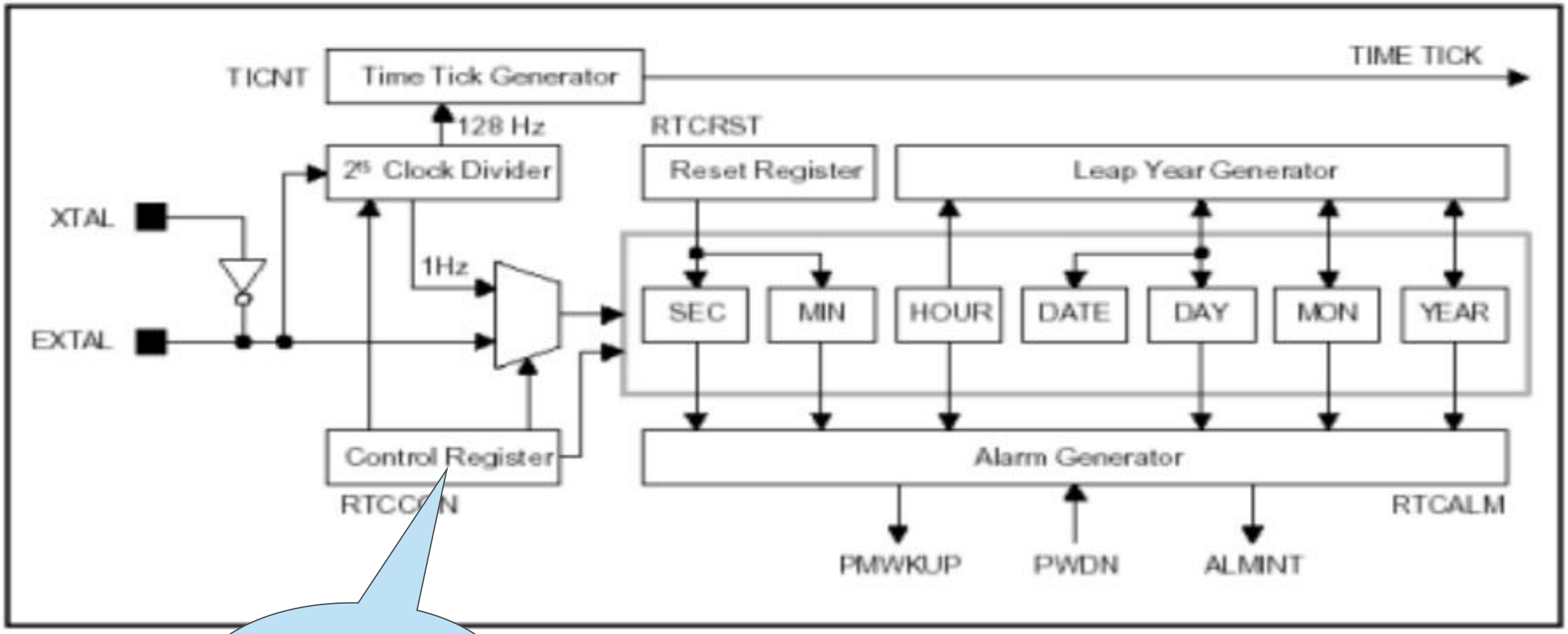
RTOS的时钟

- **时钟中断** 硬件时钟设备
  - 产生系统时钟的中断
  - 由定时/计数器产生的输出脉冲触发中断产生
- **时钟管理** 以系统时钟为基础
  - 维持日历时间 acoral_Ticks_entry()
  - 任务有限等待的计时 time_delay_deal()
  - 软定时器的定时管理 timeout_delay_deal()
  - 时间片轮转调度

硬件时钟

- 两种时钟源　　硬件时钟
  - 实时时钟　RTC；定时器/计数器
- RTC　　系统时基
  - 电池供电：系统断电，也可维持日期和时间
  - 独立于操作系统，提供一个计时标准
- 定时器/计数器
  - 实时内核控制的系统时钟
  - 最小粒度是由应用和操作系统的特点决定

三星44B0X（ARM7）芯片中的RTC



RTC控制寄存器

FCLK、HCLK、PCLK

- **FCLK** 内核时钟，最大为CPU最
- **HCLK** 总线时钟，包括USB时
- **PCLK** I/O接口时钟，常用的
  的时钟配置都以PCLK为基准
- **ARM2440的FCLK** 400MHz
- FCLK:HCLK:PCLK=1:4:8 可配置
- 定时器输出时钟频率=PCLK ÷ (prescaler+1) ÷ divider

CPU提供的时钟信号

AHB提供的时钟信号：用于高速外设

APB提供的时钟信号：用于低速外设

分频器

时间管理

- 实时内核的时间管理以**系统时钟**为基础
- **系统时钟**一般定义为整数或长整数，提供给应用程序所有和时间有关的服务
- **系统时钟**是由定时/计数器产生的输出脉冲触发中断而产生的
- 输出脉冲的周期也是定时器的一个基准周期，称为**时标**、ticks
- ticks的**大小可调**：如 5 ms产生一个中断

典型应用：μCOS的延时处理

```
void  OSTimeDly (INT16U ticks) {
    ......

    if (ticks > 0) {                                    //延时值>0
        OS_ENTER_CRITICAL();
        if  ((OSRdyTbl[OSTCBCur->OSTCBY] &
            = ~OSTCBCur->OSTCBBitX) == 0) {
            // 清优先级就绪组对应位；当前任务进入延时
            OSRdyGrp &= ~OSTCBCur->OSTCBBitY;
        }
        OSTCBCur->OSTCBDly = ticks;      //延时存入TCB
        OS_EXIT_CRITICAL();
        OS_Sched();
    }
}
```

OSRdyGrp 中序号

若在就绪队列中，则删除

典型应用：看门狗（WatchDog）

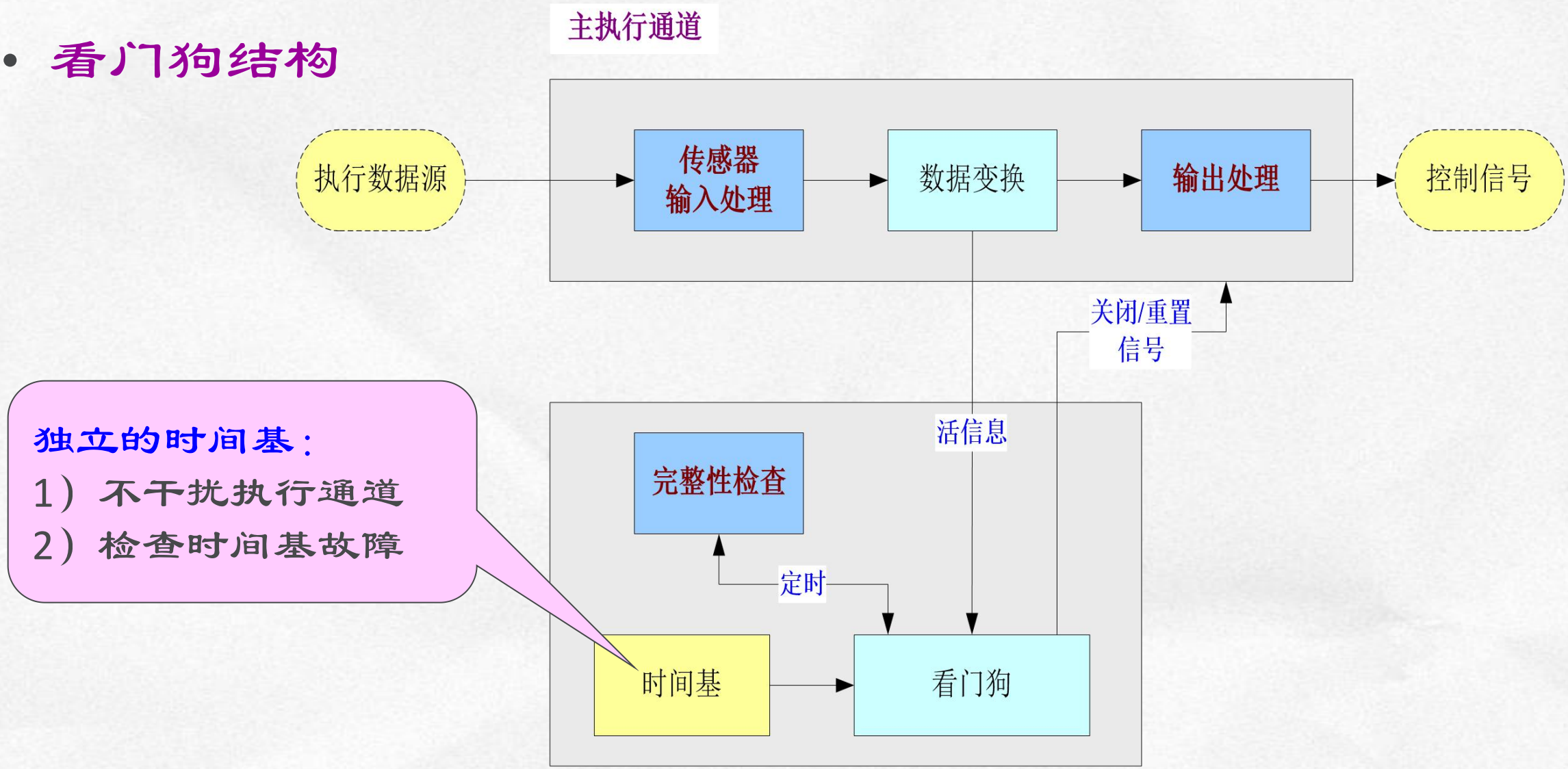
- 一个特定的时间延迟处理
- 以系统时钟中断（优先级）来完成服务代码执行
- 相关处理
  - 创建一个看门狗定时器
  - **初始化**　启动看门狗定时器，指定其延迟（Delay）和定时结束时调用的C函数（Handler）
  - **计时**　时基（**RTC/系统时钟**）为单位计时
  - **延迟到达**　调用C函数（Handler）
  - 取消看门狗定时器

典型应用：看门狗（2）

- 看门狗类别
  - 独立看门狗　iwdg，独立系统故障探测器
  - 窗口看门狗　wwdg，系统运行故障探测器
  - 相同　均用于探测系统故障：通过编写代码定时发送故障清零信号（“喂狗”），表示系统运行正常。一旦出错，“喂狗”无法执行，计数直到零并发生中断，提供相应处理（如重启）
  - 差异　lwdg有独立时钟，不受系统影响，监测硬件；wwdg使用系统时钟（系统时钟错则失效），监测软件

典型应用：看门狗（3）

- 看门狗结构



典型应用：看门狗（4）

- 喂狗：将计数值赋给WTCNT寄存器

```
static void s3c2410wdt_keepalive(void)   {
        spin_lock(&wdt_lock);
        writel(wdt_count, wdt_base + S3C2410_WTCNT);
        spin_unlock(&wdt_lock);
}
```

典型应用：看门狗（5）

- 关看门狗：清WTCON寄存器

```
static void s3c2410wdt_stop(void) {
        spin_lock(&wdt_lock);
        __s3c2410wdt_stop();
        spin_unlock(&wdt_lock);
}
```

WTCON寄存器位[5]和位[0]清零，不输出复位信号，停止看门狗

```
static void __s3c2410wdt_stop(void) {
        unsigned long wtcon;
        wtcon = readl(wdt_base + S3C2410_WTCON);
        wtcon &= ~(S3C2410_WTCON_ENABLE | S3C2410_WTCON_RSTEN);
        writel(wtcon, wdt_base + S3C2410_WTCON);
}
```

典型应用：看门狗（6）

- 启动看门狗

```
static void s3c2410wdt_start(void) {
        unsigned long wtcon;

        spin_lock(&wdt_lock);
        __s3c2410wdt_stop();          //关看门狗

        wtcon = readl(wdt_base + S3C2410_WTCON);
        // 开启看门狗，设置分频系数为128
        wtcon |= S3C2410_WTCON_ENABLE | S3C2410_WTCON_DIV128;
        if (soft_noboot) {
            wtcon |= S3C2410_WTCON_INTEN;      //值为1，一般定时器产生中断
            wtcon &= ~S3C2410_WTCON_RSTEN; // 不输出复位信号：重启无效
```

控制寄存器

典型应用：看门狗（7）

- 启动看门狗（2）

```
if (soft_noboot) {
        wtcon |= S3C2410_WTCON_INTEN;        //使能看门狗中断
        wtcon &= ~S3C2410_WTCON_RSTEN;       // 不输出复位信号
}
else {
        wtcon &= ~S3C2410_WTCON_INTEN;       //禁止看门狗中断
        wtcon |= S3C2410_WTCON_RSTEN;        //允许输出复位信号
}
writel(wdt_count, wdt_base + S3C2410_WTDAT); // 设置预分频系数和分频系数
writel(wdt_count, wdt_base + S3C2410_WTCNT); //写入计数值
writel(wtcon, wdt_base + S3C2410_WTCON);     //写WTCON寄存器，启动
spin_unlock(&wdt_lock);
}
```

数据寄存器

时钟中断

aCoral的时钟中断初始化

- ticks　　操作系统运行的时基；调度的一个激发源；决定整个系统的时间粒度
- 时钟中断初始化　　aCoral系统初始化的一项重要工作
- 初始化为位置　　开发板完成启动和aCoral的加载后

时钟HAL层初始化

- ticks时钟中断相关寄存器初始化设置

定时器控制寄存器

```
void hal_Ticks_init(){
    rTCON = rTCON & (~0xf) ;          // 清0
    rTCFG0 &= 0xFFFF00;
    rTCFG0 |= 0xF9;       //prescaler（分频器）等于249
    rTCFG1 &= ~0x0000F;
    rTCFG1 |= 0x2;        //divider等于8，定时器4的时钟频率为25kHz
    // 定时器4的中断周期
    rTCNTB0 = PCLK /(8*(249+1)*ACORAL_TickS_PER_SEC);
    rTCON = rTCON & (~0xf) |0x02;          //手动更新Timer0
    rTCON = rTCON & (~0xf) |0x09;          //启动定时器
}
```

时钟内核层初始化

```
void acoral_Ticks_init(){
        ticks=0;                                    //初始化滴答时钟计数器
        //设置Ticks处理函数
        acoral_intr_attach(HAL_TickS_INTR,acoral_Ticks_entry);
        hal_Ticks_init();                           //用于ticks的时钟初始化
        acoral_intr_unmask(HAL_TickS_INTR);
        return;
}
```

ticks处理程序：
hal中断号换为内核中断号
对实时：调实时isr

绑定Ticks处理程序

```
acoral_32 acoral_intr_attach(acoral_vector vector,
                                void (*isr)(acoral_vector)){
    acoral_vector index;
    HAL_TRANSLATE_VECTOR(vector,index);
    if(intr_table[index].type!=ACORAL_RT_[illegible]
        intr_table[index].isr =isr;
    else
        HAL_INTR_ATTACH(vector,isr);
    return 0;
}
```

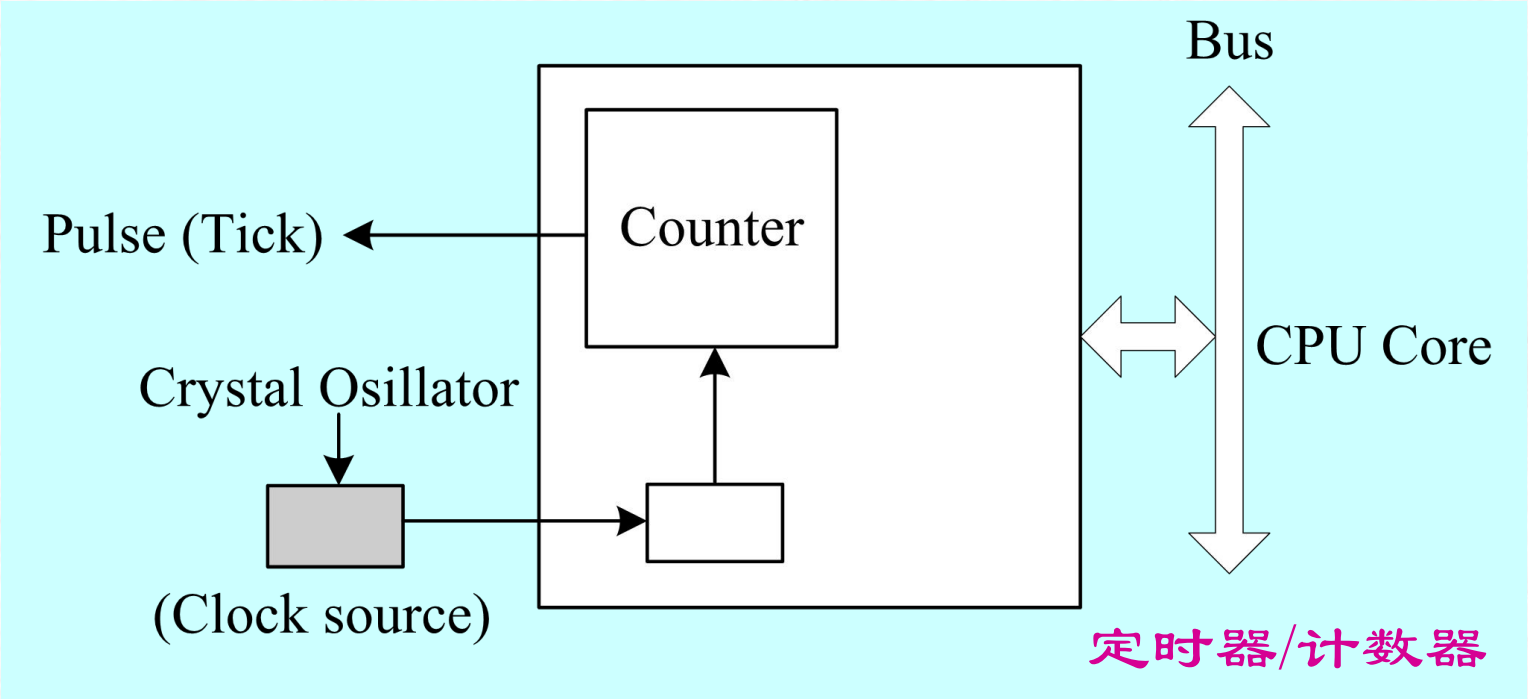
hal中断号换为内核中断号

实时中断服务处理程序：中断发生，立即处理

时钟管理

时钟管理概述

- 不同RTOS中，RTC和系统时钟之间关系不同：决定了RTOS的时钟运行机制
- 以RTC为基准：实时内核用RTC初始化系统时钟，并维护系统时钟
- 实时内核的时间管理：以系统时钟为基础

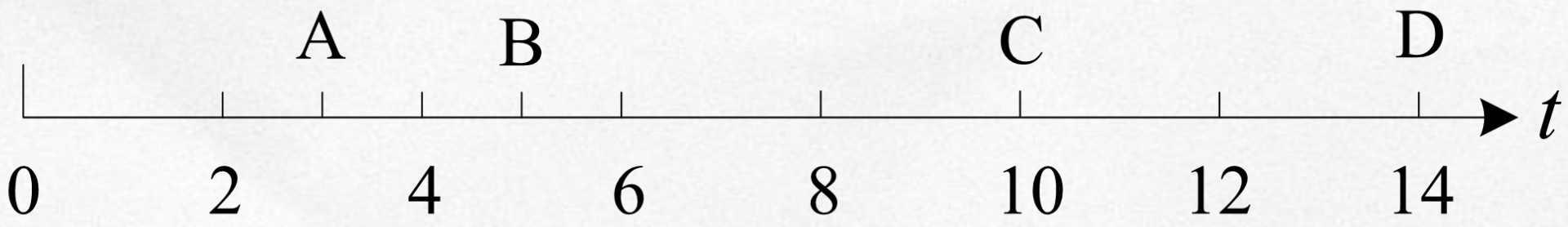


时钟管理概述（2）

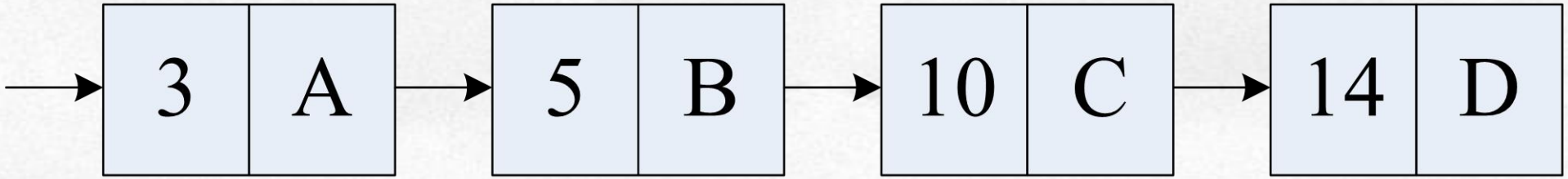
- 时间过程：定时器→脉冲信号→时钟中断→HAL层→内核层→ticks处理acoral_Ticks_entry
- 时间片：时钟中断→ticks处理：任务执行时间+1 → 时间片到？ → 调度
- 延迟操作：任务挂起→延时设置→时钟中断→ticks处理：延时-1 → 延时=0？→重就绪
- 定时管理：……

时间等待队列

- 时间相关操作的基本数据结构
- 延时/等待要求

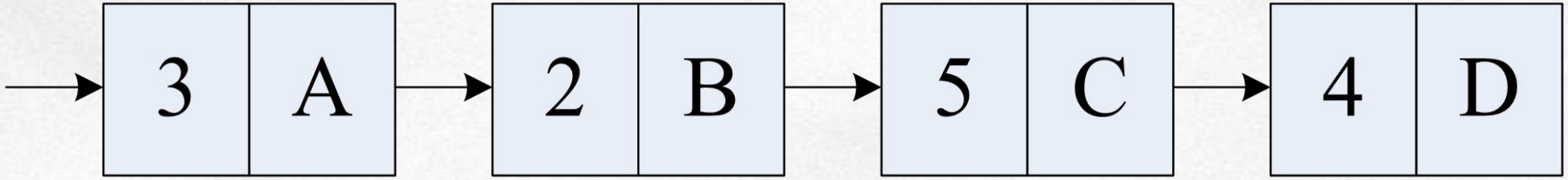


- 时间等待队列：绝对时间

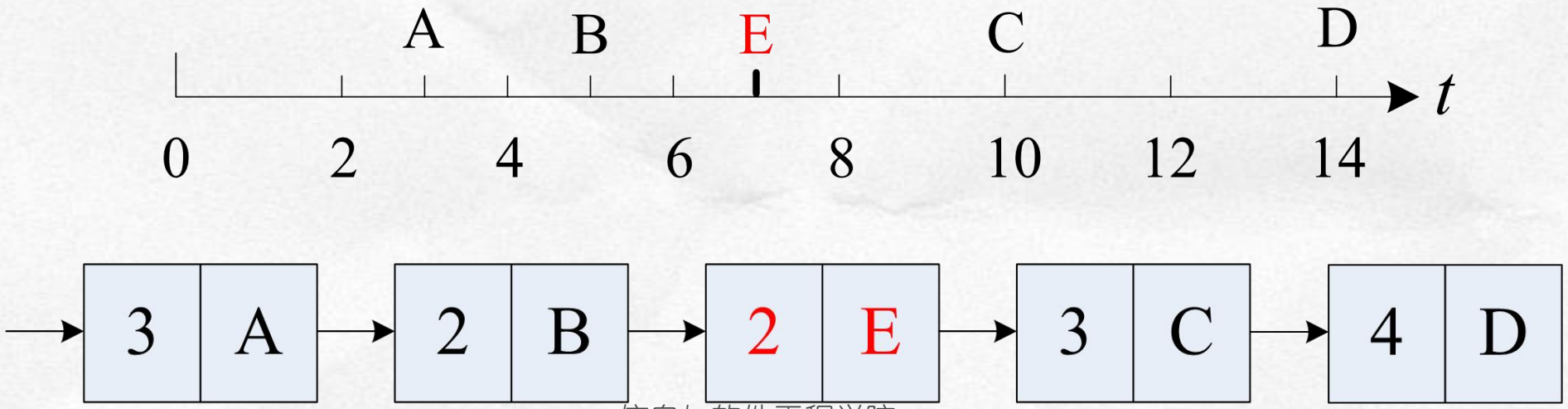


时间等待队列（2）

- 队列变化：相对时间 —— 差分链表



- 新等待的加入：相对时间



延时处理程序

```
void time_delay_deal(){
    acoral_list_t   *tmp,*tmp1,*head;
    acoral_thread_t *thread;
    head = &time_delay_queue.head;
    if(acoral_list_empty(head))          //时间等待链中无当前任
        return;
    // 表头任务对应的TCB地址
    thread=list_entry(head->next,acoral_thread_t,waiting);
    ACORAL_ASSERT(thread,"in time deal");
    thread->delay--;                     //延时减一
    for(tmp=head->next;tmp!=head;){
        thread=list_entry(tmp,acoral_thread_t,waiting);
        ACORAL_ASSERT(thread,"in time deal for");
        if(thread->delay>0)
            break;
```

断言：调试用
检查任务合法性

延时处理程序（2）

```
    // 防止add判断delay时取下thread
    ~~#ifndef CFG_TickS_PRIVATE~~
        ~~acoral_spin_lock(&head->lock)~~;
        ~~acoral_spin_lock(&tmp->lock)~~;
    ~~#endif~~
    tmp1=tmp->next;
    acoral_list_del(&thread->waiting);                    //退出时间队列
    ~~#ifndef CFG_TickS_PRIVATE~~
        ~~acoral_spin_unlock(&tmp->lock~~);
        ~~acoral_spin_unlock(&head->lo~~ck);
    ~~#endif~~
    tmp=tmp1;
    thread->state&=~ACORAL_THREAD_STATE_DELAY;            //状态切换为就绪
    acoral_rdy_thread(thread);                            //挂就绪队列
    }
}
```

练习题

1、结合代码，说明HAL_INTR_ENTRY的在中断响应过程中的作用？它是如何区分不同中断源的？该函数执行过程中进行了几次模式切换?为什么要进行切换？

```
stmfd   sp!,   {r0-r12,lr}
mrs     r1,    spsr
stmfd   sp!,   {r1}
msr     cpsr_c, #SVCMODE|NOIRQ
stmfd   sp!,   {lr}
ldr     r0,    =INTOFFSET
ldr     r0,    [r0]
mov     lr,    pc
ldr     pc,    =hal_all_entry
ldmfd   sp!,   {lr}
msr     cpsr_c,#IRQMODE|NOINT
ldmfd   sp!,{r0}
msr     spsr_cxsf,r0
ldmfd   sp!,{r0-r12,lr}
subs    pc,lr,#4
```

2、aCoral硬件抽象层的中断号与内核层的中断号有什么不同？为什么会有不同？内核是如何实现硬件抽象层中断号到内核层中断号的映射？

谢谢！